# BLIND 75 CHEAT SHEET - JAVA PATTERNS & EXAMPLES

==========

## Arrays & Hashing

*Use HashMap for frequency count or checking complements.*

```
Map<Integer, Integer> map = new HashMap<>();
for (int num : nums) {
map.put(num, map.getOrDefault(num, 0) + 1);
}
```

## Two Pointers

*Common in sorted arrays or linked lists for pair problems.*

```
int left = 0, right = nums.length - 1;
while (left < right) {
int sum = nums[left] + nums[right];
if (sum == target) return true;
else if (sum < target) left++;
else right--;
}
```

## Sliding Window

*Used to track a window of elements in a linear array.*

```
int max = 0, sum = 0, k = 3;
for (int i = 0; i < nums.length; i++) {
sum += nums[i];
if (i >= k) sum -= nums[i - k];
max = Math.max(max, sum);
}
```

## Stack

*Useful for Next Greater Element, Valid Parentheses, etc.*

```
Stack<Character> stack = new Stack<>();
for (char c : s.toCharArray()) {
if (c == '(') stack.push(c);
else if (stack.isEmpty() || stack.pop() != '(') return false;
}
return stack.isEmpty();
```

## Binary Search

*Always on sorted arrays or to minimize/maximize answers.*

```
int l = 0, r = nums.length - 1;
while (l <= r) {
int mid = (l + r) / 2;
if (nums[mid] == target) return mid;
else if (nums[mid] < target) l = mid + 1;
else r = mid - 1;
}
```

## Linked List

*Use fast and slow pointers to detect cycles.*

```
ListNode slow = head, fast = head;
while (fast != null && fast.next != null) {
slow = slow.next;
fast = fast.next.next;
if (slow == fast) return true;
}
```

## Trees

*Use recursion for DFS on binary trees.*

```
void inorder(TreeNode root) {
if (root == null) return;
inorder(root.left);
System.out.print(root.val);
inorder(root.right);
}
```

## Tries

*Used for word prefix matching.*

```
class TrieNode {
TrieNode[] children = new TrieNode[26];
boolean isWord = false;
}
```

## Heap / Priority Queue

*Used for Kth largest/smallest, sorting by priority.*

```
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
```

```
for (int num : nums) {
minHeap.offer(num);
if (minHeap.size() > k) minHeap.poll();
}
```

## Backtracking

*Used in permutations, subsets, N-Queens, Sudoku.*

```
void backtrack(List<List<Integer>> res, List<Integer> temp, int[] nums, boolean[] used) {
if (temp.size() == nums.length) res.add(new ArrayList<>(temp));
for (int i = 0; i < nums.length; i++) {
if (used[i]) continue;
used[i] = true;
temp.add(nums[i]);
backtrack(res, temp, nums, used);
used[i] = false;
temp.remove(temp.size() - 1);
}
}
```

## Graphs (BFS/DFS)

*Use BFS for shortest path, DFS for full traversal.*

```
Queue<Integer> q = new LinkedList<>();
Set<Integer> visited = new HashSet<>();
q.offer(start);
while (!q.isEmpty()) {
int node = q.poll();
if (!visited.contains(node)) {
visited.add(node);
for (int neighbor : graph.get(node)) {
q.offer(neighbor);
}
}
}
```

## Dynamic Programming

*Used when subproblems repeat, use memo or bottom-up table.*

```
int[] dp = new int[n+1];
dp[0] = 1;
for (int i = 1; i <= n; i++) {
dp[i] = dp[i-1] + dp[i-2];
}
```

## Greedy

*Make locally optimal choices (Interval Scheduling, Jump Game).*

```
int end = 0, farthest = 0, jumps = 0;
for (int i = 0; i < nums.length - 1; i++) {
farthest = Math.max(farthest, i + nums[i]);
if (i == end) {
jumps++;
end = farthest;
}
}
```

## Intervals

*Sort by start or end time and merge or count overlaps.*

```
Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
List<int[]> merged = new ArrayList<>();
for (int[] interval : intervals) {
if (merged.isEmpty() || merged.get(merged.size()-1)[1] < interval[0]) {
merged.add(interval);
} else {
merged.get(merged.size()-1)[1] = Math.max(merged.get(merged.size()-1)[1], interval[1]);
}
}
```

## Bit Manipulation

*Use XOR to cancel out duplicate numbers.*

```
int res = 0;
for (int num : nums) res ^= num;
return res;
```

## Prefix Sum

*Build a running total array to quickly find ranges.*

```
int[] prefix = new int[nums.length+1];
for (int i = 0; i < nums.length; i++) {
prefix[i+1] = prefix[i] + nums[i];
}
int sumRange = prefix[j+1] - prefix[i];
```

## Union Find (Disjoint Set)

*Use for connected components or cycle detection.*

```
int[] parent = new int[n];
for (int i = 0; i < n; i++) parent[i] = i;

int find(int x) {
if (parent[x] != x) parent[x] = find(parent[x]);
return parent[x];
}

void union(int x, int y) {
parent[find(x)] = find(y);
}
```

End of Cheat Sheet. Happy Coding!